

Machine Learning pour le secteur bancaire

De la donnée au modèle en production : apprentissage supervisé et non supervisé

Formation — experts métier & IT banking

Labs : Credit Scoring (classification) • Segmentation client (clustering)

Programme de la formation

1. Introduction : IA, ML, DL et IA générative

2. Cycle de vie d'un projet ML & aperçu MLOps

3. Apprentissage supervisé

4. Apprentissage non supervisé : clustering

5. Labs, applications & conclusion

Objectifs & public

À l'issue de la formation, vous saurez

- situer IA, ML, DL et IA générative
- reconnaître le **bon type d'apprentissage** pour un besoin métier
- dérouler le **cycle de vie** complet d'un projet ML, jusqu'à la production
- comprendre les principaux **modèles**, leurs forces et leurs limites
- choisir les bonnes **métriques** et diagnostiquer l'**overfitting**
- challenger un projet ML (data, risque, conformité)

Public

Experts métier et IT du secteur bancaire.

Aucun prérequis en mathématiques avancées : les notions sont présentées de manière intuitive.

Pratique

Lab 1 — Credit scoring (classification)
Lab 2 — Segmentation client (clustering)

+ 2 applications FastAPI / Streamlit avec dashboards KPIs

Introduction : IA, ML, DL et IA générative

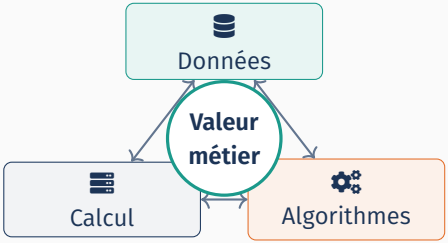
Pourquoi maintenant ? Pourquoi la banque ?

Trois moteurs réunis

- **Données** : transactions, crédits, interactions, documents
- **Calcul** : cloud, GPU, calcul distribué
- **Algorithmes** open source matures (scikit-learn, XGBoost, PyTorch, LLM)

La banque est une industrie de la donnée

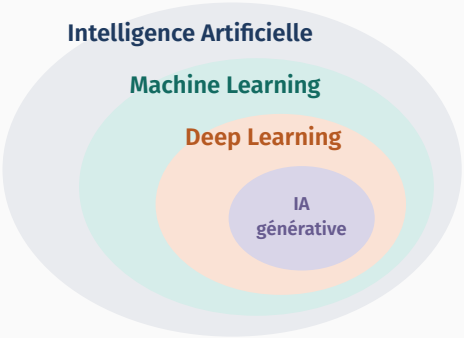
- décisions répétées à grande échelle
- coût élevé des erreurs (pertes, fraude)
- exigence réglementaire (explicabilité, audit)



Cas bancaire

Scoring, fraude, lutte anti-blanchiment, churn, segmentation, recommandation, automatisation documentaire...

IA, Machine Learning, Deep Learning, IA générative



IA Systèmes qui réalisent des tâches demandant « de l'intelligence » (règles expertes, planification, apprentissage…)

ML Algorithmes qui **apprennent des règles à partir d'exemples** au lieu de les programmer

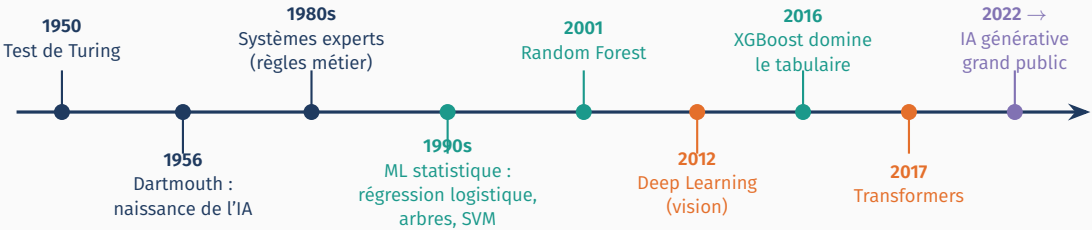
DL ML à base de **réseaux de neurones profonds** : images, texte, voix

IAG Modèles de DL qui **génèrent** du contenu (texte, code, images) : LLM type GPT, Claude, Mistral…

À retenir

Sur les **données tabulaires** bancaires (clients, comptes, crédits), le **ML « classique »** reste la référence.

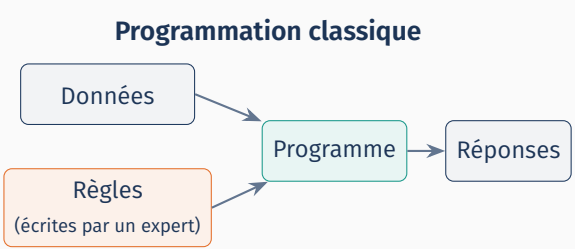
Quelques jalons

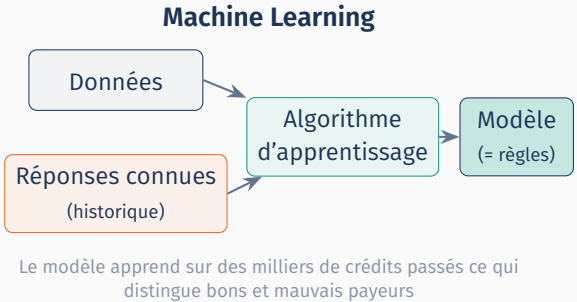


🏛 Cas bancaire

La banque utilise des modèles statistiques depuis longtemps (**scores de crédit** par régression logistique, années 1980-90). Le ML moderne prolonge cette culture : **mêmes exigences** de robustesse, de validation et d'explicabilité.

Programmation classique vs Machine Learning





Quand le ML est pertinent

Règles trop nombreuses ou évolutives, beaucoup d'exemples historiques, décision répétée à grande échelle

Quand il ne l'est pas

Peu de données, règle réglementaire stricte et simple, besoin de déterminisme absolu

Vocabulaire de base

Variables explicatives (features) X — Cible y

1 ligne = 1 observation (un client)

Client	Âge	Durée	Montant	Compte courant	Défaut
C001	42	24	5 000	débiteur	1
C002	35	12	1 500	sans compte	0
C003	28	36	9 200	0–200	0
...	...	...	...	...	...
C999	51	6	800	≥ 200	?

Entraînement

Le modèle apprend la relation $X \to y$ sur l'historique

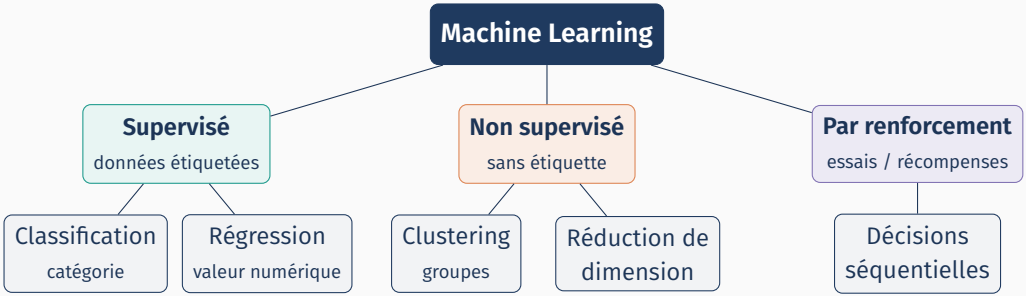
Inférence

Le modèle prédit y pour un **nouveau** client (C999)

Modèle

Fonction $\hat{y} = f(X)$ dont les **paramètres** sont appris

Les grandes familles d'apprentissage



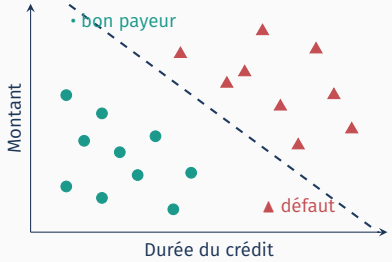
« Ce client va-t-il faire défaut ? »
on connaît la réponse pour le passé

« Quels profils de clients avons-nous ? »
pas de réponse attendue

« Quelle action maximise le gain à long terme ? »
on apprend en interagissant

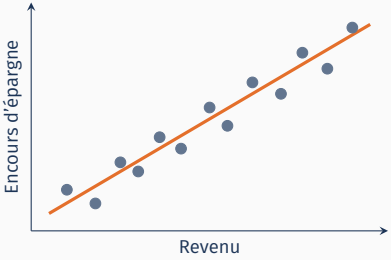
Apprentissage supervisé : classification & régression

Classification : prédire une **catégorie**



Défaut oui/non · fraude · churn · catégorie de réclamation

Régression : prédire une **valeur numérique**



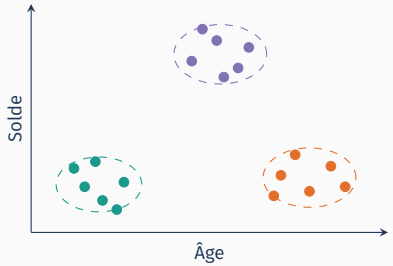
Montant de perte (LGD) · valeur d'un bien · flux de trésorerie · CLV

À retenir

Même démarche dans les deux cas : apprendre sur un historique où la **bonne réponse est connue**, puis généraliser à de nouveaux cas.

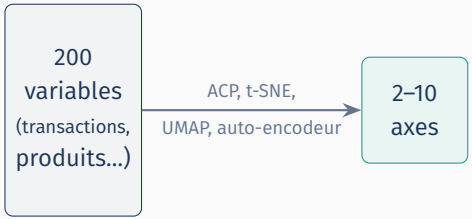
Apprentissage non supervisé : clustering & réduction de dimension

Clustering : découvrir des **groupes**



Segmentation clients · détection d'atypiques (fraude, AML)

Réduction de dimension : **résumer**



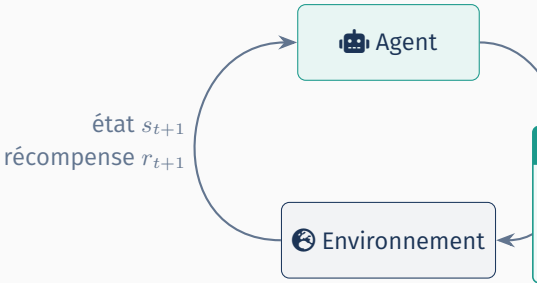
Visualisation · compression · débruitage · préparation à d'autres modèles

Pas de « bonne réponse »

La qualité se juge à l'**utilité métier** : segments interprétables, actionnables, stables.

Apprentissage par renforcement

- L'agent apprend une **stratégie** (politique) par **essais et erreurs**
- Objectif : maximiser la **récompense cumulée** à long terme
- Pas d'historique étiqueté : on apprend en **interagissant**



Cas bancaire

Exécution optimale d'ordres, tarification dynamique, stratégies de relance en recouvrement, allocation de portefeuille (recherche).

Encore peu déployé en banque : besoin d'un environnement simulable, risques de comportements imprévus.
Utilisé aussi pour aligner les LLM (RLHF).

Auto-supervision & IA générative

Comment apprend un LLM ?

- **Auto-supervision** : l'étiquette est dans la donnée elle-même
  « Le client souhaite clôturer son _____ »
  → prédire le mot suivant
- Pré-entraînement sur d'énormes corpus, puis **ajustement** (fine-tuning, RLHF)
- Utilisation : **prompt**, **RAG** (réponses ancrées dans les documents de la banque), **agents** (outils)

Points de vigilance

Hallucinations · confidentialité des données · coût · traçabilité · dépendance fournisseur

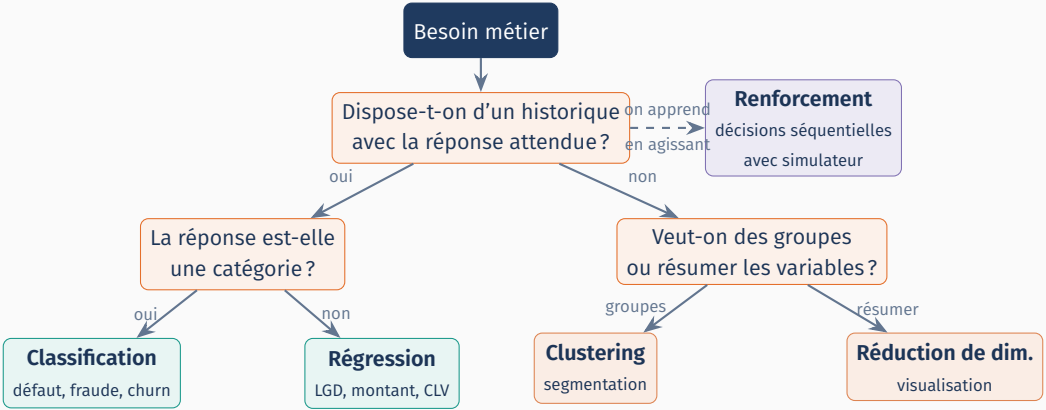
Cas d'usage IAG en banque

- analyse de documents KYC, contrats, bilans
- assistant conseiller (procédures)
- synthèse de réclamations, d'appels
- aide au développement de code

À retenir

IAG pour le **texte et les documents** ; ML classique pour les **décisions sur données tabulaires**.

Quel type d'apprentissage pour quel besoin ?



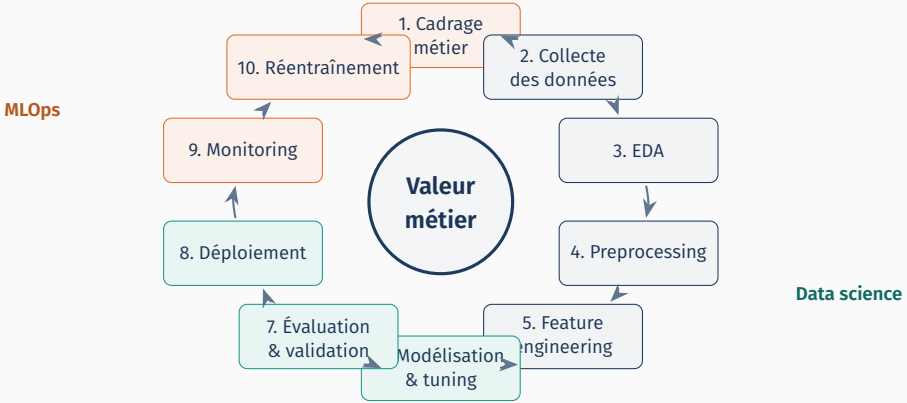
Cartographie des cas d'usage bancaires

Domaine	Supervisé	Non supervisé	IA générative
Risque de crédit	Score d'octroi (PD), LGD, alertes précoces	Segmentation de portefeuille	Analyse de bilans, notes de crédit
Fraude & AML	Détection de fraude carte, scoring de transactions	Détection d'atypiques, réseaux suspects	Synthèse d'alertes, rédaction de déclarations
Marketing & client	Churn, appétence produit, next best offer	**Segmentation client**, personas	Assistant conseiller, personnalisation
Opérations	Prévision de flux, catégorisation de transactions	Regroupement de réclamations	Traitement documentaire, KYC
Marchés	Prévision, pricing	Régimes de marché	Synthèse de recherche

🧪 Nos deux labs : **credit scoring** (classification) et **segmentation client** (clustering).

Cycle de vie d’un projet ML & aperçu MLOps

Le cycle de vie complet d'un projet ML



🔁 Un projet **itératif** : le modèle en production est un **produit vivant**, pas un livrable figé.

Étape 1 — Cadrage métier : la plus importante

Les questions à se poser

- Quelle **décision** le modèle va-t-il éclairer ? Qui l'utilise ?
- Quelle est la **cible** exacte ? (« défaut » = 90 jours d'impayés sur 12 mois ?)
- Quel **KPI métier** améliorer ? (pertes, taux d'acceptation, coût de traitement)
- Quel est le **coût des erreurs** ? Sont-elles symétriques ?
- Quelles **données** sont disponibles **au moment de la décision** ?
- Quelles **contraintes** : explicabilité, latence, réglementation ?

🏛 Exemple : score d'octroi

Décision : accepter / revoir / refuser une demande de crédit conso

Cible : défaut dans les 12 mois

Coût : accepter un mauvais payeur $\approx$ **5×** refuser un bon

Contraintes : décision motivée au client, modèle validé par le risque, revue annuelle

Piège classique

Optimiser une métrique technique (accuracy) qui ne reflète pas le besoin métier.

Étapes 2–3 — Données : collecte, qualité, exploration

Sources typiques en banque

- Core banking : comptes, soldes, produits
- Transactions (cartes, virements)
- CRM, interactions, réclamations
- Données externes : bureaux de crédit, données macro

Dimensions de la qualité

- complétude · exactitude · cohérence · fraîcheur · traçabilité

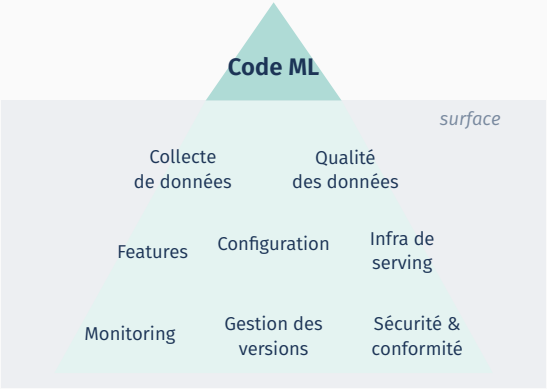
EDA : analyse exploratoire

- distributions, valeurs manquantes, valeurs extrêmes
- relation de chaque variable avec la cible
- corrélations, redondances
- **confrontation à l'expertise métier**

Fuite d'information temporelle

N'utiliser que des données **disponibles au moment de la décision** (pas d'info postérieure à l'octroi !).

Le modèle n'est que la partie visible



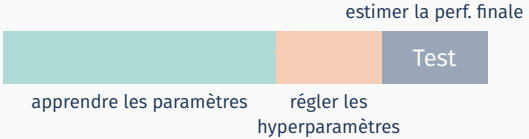
- Dans un système ML réel, le code du modèle est une **petite fraction** du système
- Le reste : pipelines de données, tests, déploiement, supervision, gouvernance
- D'après Sculley et al., *Hidden Technical Debt in Machine Learning Systems*, NeurIPS 2015

À retenir

La préparation des données et l'industrialisation représentent **l'essentiel de l'effort** d'un projet.

Étapes 6–7 — Modélisation, évaluation, validation

Découpage des données



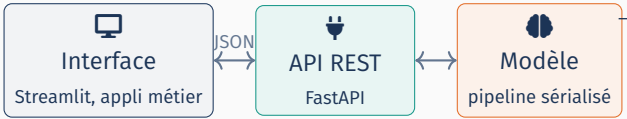
Le jeu de **test** n'est utilisé **qu'une fois**, à la fin.
En banque, on ajoute souvent un échantillon **hors-période** (out-of-time) : les données les plus récentes.

🏛 Validation de modèle

- **validation indépendante** par une équipe distincte des développeurs
- documentation : données, choix, limites, tests de performance et de stabilité
- inventaire des modèles, revue périodique
- cadre de **gestion du risque de modèle**

Étape 8 — Déploiement : mettre le modèle au service du métier

Architecture 3-tiers (celle de nos labs)



L'API est le **contrat** entre le modèle et les applications : on peut changer de modèle sans toucher aux applications.

Mode	Exemple bancaire
Batch	Re-scoring mensuel du portefeuille, segmentation
Temps réel	Octroi en ligne, fraude carte (millisecondes)
Streaming	Alertes AML sur flux de transactions
Embarqué	Application mobile

À retenir

On déploie le **pipeline complet** (préparation + modèle) : en production, il reçoit les **données brutes**, exactement comme à l'entraînement.

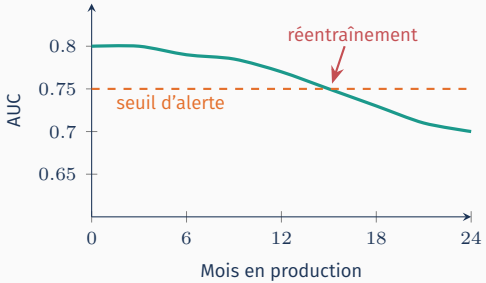
Étape 9 — Monitoring : un modèle se dégrade

Pourquoi ? Conjoncture, nouvelle offre, nouveau canal, fraudeurs qui s'adaptent…

- **Data drift** : la distribution des variables change (clients plus jeunes)
- **Concept drift** : la relation variables → cible change (plus de défauts en récession)

Que surveiller ?

- données d'entrée et scores produits
- performance réelle (quand la cible est connue)
- KPI métier, latence, erreurs techniques



Délai de la cible

En crédit, la performance réelle n'est connue qu'avec retard : on surveille donc d'abord la **stabilité des données et des scores**.

Mesurer la dérive : le Population Stability Index (PSI)

Comparer la répartition actuelle d'une variable (ou du score) à sa répartition de référence :

$$\text{PSI} = \sum_i \left(\%_{\text{actuel},i} - \%_{\text{réf},i}\right) \times \ln \frac{\%_{\text{actuel},i}}{\%_{\text{réf},i}}$$

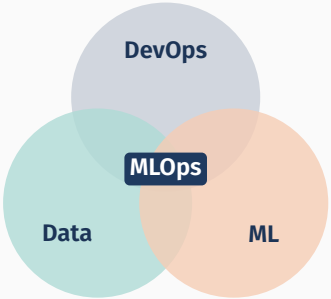
PSI	Lecture	Action
$< 0{,}10$	stable	RAS
$0{,}10 - 0{,}25$	dérive modérée	analyser
$> 0{,}25$	dérive significative	revoir le modèle



Exemple : PSI $\approx 0{,}13 \rightarrow$ dérive modérée

🏛 Indicateur standard du suivi des modèles de crédit — utilisé dans les applications des deux labs.

Qu’est-ce que le MLOps ?



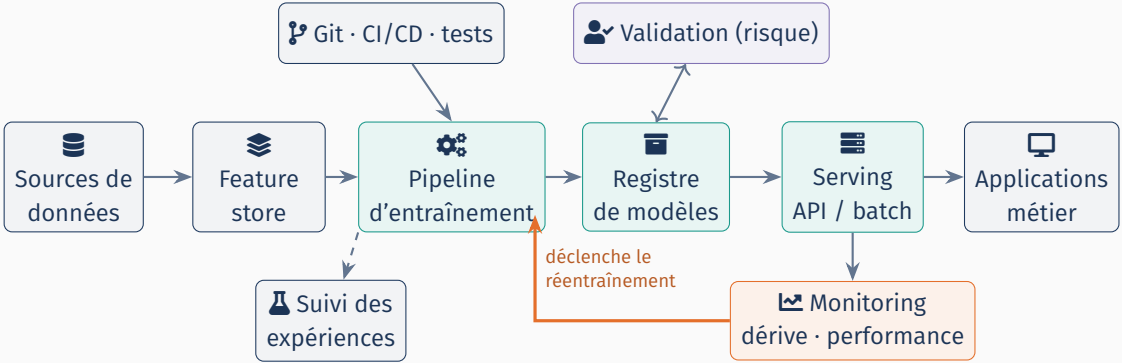
Pratiques pour mettre en production et maintenir des modèles de façon fiable et automatisée.

- **Versionner** code, données et modèles
- **Tracer** les expériences (paramètres, métriques)
- **Automatiser** entraînement, tests et déploiement (CI/CD/CT)
- **Surveiller** en continu et **réentraîner**
- **Gouverner** : registre de modèles, approbations, audit

À retenir

Du « notebook qui marche sur mon poste » à un **service fiable, reproductible et auditable**.

Une chaîne MLOps type



CI — intégration continue
tests du code et des données

CD — déploiement continu
mise en production automatisée

CT — entraînement continu
réentraînement déclenché

Niveaux de maturité MLOps

	Niveau 0 — Manuel	Niveau 1 — Pipeline automatisé	Niveau 2 — CI/CD/CT complet
Entraînement	Notebooks, scripts manuels	Pipeline reproductible, déclenchable	Pipeline testé, versionné, déclenché automatiquement
Déploiement	Remise d'un fichier modèle à l'IT	Déploiement du pipeline	Déploiement automatisé après validation
Monitoring	Ponctuel, ou absent	Suivi de dérive et de performance	Alertes, réentraînement automatique
Mise à jour	Annuelle ou moins	Mensuelle / trimestrielle	Selon les besoins, en continu

À retenir

En banque, l'automatisation reste encadrée : un **humain valide** chaque nouvelle version d'un modèle réglementé.

L’outillage MLOps

Fonction	Outils courants
Versionnement	Git, DVC (données), registres d’artefacts
Suivi d’expériences & registre	MLflow, Weights & Biases
Orchestration de pipelines	Airflow, Kubeflow, Prefect, Dagster
Feature store	Feast, solutions des plateformes cloud
Serving	FastAPI, BentoML, Seldon, KServe, conteneurs Docker / Kubernetes
Monitoring	Evidently, Prometheus + Grafana, outils des plateformes
Plateformes intégrées	Azure ML, AWS SageMaker, Google Vertex AI, Databricks, Dataiku

Dans nos labs

scikit-learn **Pipeline** + **joblib** (sérialisation) + **FastAPI** (API) + **Streamlit** (interface et dashboards) + **PSI** (monitoring).

Contraintes spécifiques au secteur bancaire

Gestion du risque de modèle

Inventaire, validation indépendante, documentation, revue périodique, suivi des performances, plan de remédiation.

Explicabilité

Capacité à expliquer une décision **individuelle** (refus de crédit) et le comportement **global** du modèle.

Données personnelles (RGPD / nLPD)

Minimisation, finalité ; encadrement des décisions **entièrement automatisées** (art. 22 RGPD).

AI Act (UE)

Évaluer la solvabilité de personnes physiques = système **« à haut risque »** : qualité des données, documentation, transparence, **contrôle humain**, robustesse.

Superviseurs

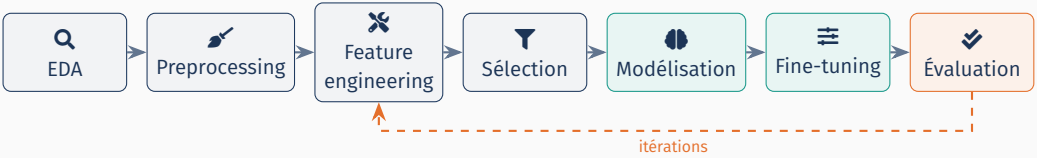
Orientations de l'ABE sur l'octroi de crédit (modèles automatisés) ; communication FINMA 08/2024 sur la gouvernance des risques liés à l'IA.

À retenir

Équité : mesurer les écarts entre groupes, y compris via des variables **proxy**.

Apprentissage supervisé

La démarche supervisée de bout en bout



Fil rouge : le Lab 1 — credit scoring

German Credit (UCI) : 1 000 demandes de crédit, 20 variables (7 numériques, 13 catégorielles)
Cible : **défaut** (30 %) vs bon payeur (70 %)

Cinq modèles comparés

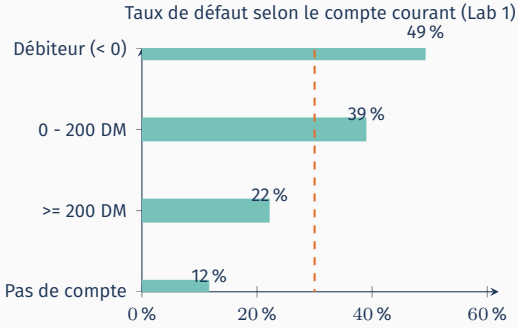
Régression logistique · KNN · Arbre de décision · Random Forest · XGBoost

EDA : comprendre avant de modéliser

- **Univariée** : distribution de chaque variable, valeurs rares, asymétries
- **Bivariée** : lien de chaque variable avec la **cible**
- **Multivariée** : corrélations, redondances
- **Qualité** : manquants, doublons, incohérences

À retenir

Toujours **confronter à l'expertise métier**. Dans le Lab 1, un historique « critique » affiche le taux de défaut le **plus bas** : ces clients ont des crédits ailleurs, déjà filtrés par d'autres banques.



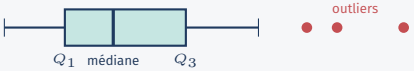
Valeurs manquantes & valeurs extrêmes

Valeurs manquantes

- **Supprimer** les lignes : seulement si peu nombreuses et aléatoires
- **Imputer** : médiane (numérique), mode ou modalité « inconnu » (catégorielle)
- Ajouter un **indicateur de manque** : l'absence est souvent une information (revenu non déclaré)
- L'imputation s'ajuste sur le **train uniquement**

Valeurs extrêmes (outliers)

- Règle IQR : hors de $[Q_1 - 1{,}5\,\text{IQR}; Q_3 + 1{,}5\,\text{IQR}]$
- **Erreur** de saisie $\rightarrow$ corriger ou supprimer
- Valeur **réelle** (gros crédit) $\rightarrow$ garder, transformer (log) ou plafonner



Cas bancaire

Lab 1 : aucune valeur manquante ; les gros montants et longues durées sont **réels et informatifs** : on les garde.

Encoder les variables catégorielles

One-hot encoding : une colonne binaire par modalité

Logement	own	rent	for free
own	1	0	0
rent	0	1	0
for free	0	0	1

Encodage ordinal : si un ordre existe

Ancienneté : < 1 an → 0 ; 1–4 ans → 1 ; 4–7 ans → 2 ; > 7 ans → 3

Points d'attention

- Beaucoup de modalités → beaucoup de colonnes
  (Lab 1 : 20 variables → 61 colonnes)
- **Modalités rares** : les regrouper (« autre »)
- Modalité **inconnue** en production : ne pas planter (`handle_unknown`)

🏛 Scoring traditionnel : modalités regroupées en classes de risque homogènes (**WoE**).

Mettre les variables à la même échelle

Standardisation (StandardScaler)

$$z = \frac{x - \mu}{\sigma}$$ moyenne 0, écart-type 1

Normalisation min-max

$$x' = \frac{x - x_{\min}}{x_{\max} - x_{\min}} \in [0, 1]$$

Sans mise à l'échelle, une variable en milliers (montant) **écrase** une variable en unités (âge) dans le calcul des distances.

Modèle	Scaling ?
KNN	indispensable
Régression logistique	recommandé
Réseaux de neurones, SVM	indispensable
Arbres, RF, XGBoost	inutile
K-Means (clustering)	indispensable

À retenir

Moyenne et écart-type sont calculés sur le **train**, puis appliqués tels quels au test et en production.

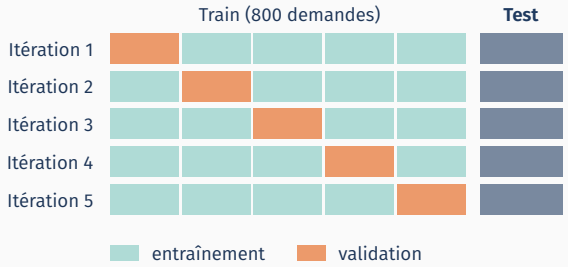
Découper les données : train / test et validation croisée

Split train / test stratifié

- 80 % train, 20 % test (Lab 1 : 800 / 200)
- **Stratifié** : même taux de défaut (30 %) dans chaque partie
- Le test est mis de côté jusqu'à la fin

Validation croisée à k plis

- le train est découpé en k parties
- on entraîne k fois, en validant à chaque fois sur une partie différente
- score = **moyenne ± écart-type**



Cas bancaire

Données temporelles : préférer une validation **chronologique** (out-of-time) pour simuler l'usage réel.

Le piège n°1 : la fuite de données (data leakage)

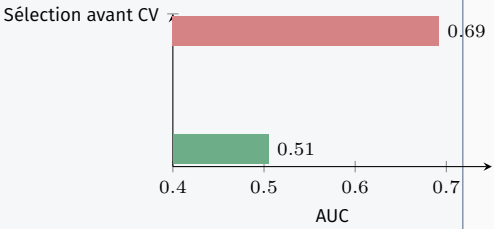
Définition : une information indisponible au moment de la prédiction « fuit » dans l'entraînement → performance **illusoire**.

Exemples bancaires

- variable postérieure à l'octroi : « nb de relances de recouvrement »
- scaling / imputation / sélection calculés **sur tout le jeu**
- même client présent dans le train et le test
- données futures dans une validation non chronologique

Expérience du Lab 1

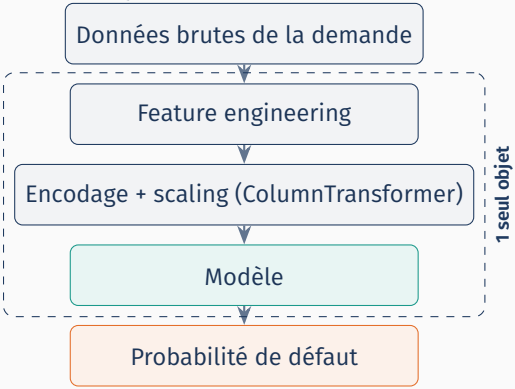
1 000 clients, 2 000 variables de **bruit pur**, cible aléatoire.
On sélectionne les 20 « meilleures » variables, puis validation croisée :



AUC de 0,69 **sur du hasard** sans pipeline ; 0,50 (la vérité) avec.

La solution : le Pipeline

Toutes les étapes qui **apprennent** des données sont chaînées et ajustées **sur le train uniquement**, à chaque pli de validation.




```
pipeline = Pipeline([
  ("fe",  FunctionTransformer(add_features)),
  ("pre", ColumnTransformer([
      ("num", StandardScaler(), num_cols),
      ("cat", OneHotEncoder(
           handle_unknown="ignore"), cat_cols),
  ])),
  ("model", LogisticRegression(C=0.03)),
])
pipeline.fit(X_train, y_train)
pipeline.predict_proba(new_application)
```

À retenir

Le pipeline est **l'objet déployé** en production.

Feature engineering : traduire l'expertise métier en variables

Technique	Exemple bancaire	Intuition
Ratios	mensualité / revenu (taux d'effort), encours / revenu	la capacité de remboursement compte plus que le montant brut
Agrégats temporels	solde moyen sur 3 / 6 / 12 mois, nb d'incidents sur 12 mois	résumer un historique de transactions
Tendances	évolution du solde, de l'épargne	un client qui se dégrade
Ancienneté	durée de la relation, âge du compte	stabilité
Discrétisation	tranches d'âge, de revenu	effets non linéaires, lisibilité
Transformations	$\log(1 + \text{montant})$	réduire l'asymétrie
Interactions	montant × durée	effets combinés

À retenir

Sur données réelles : souvent **le premier levier de performance**, devant le choix du modèle.

Feature engineering dans le Lab 1

Variable créée	Construction
`monthly_payment`	montant / durée
`log_credit_amount`	$\log(1 + \text{montant})$
`employment_years`	ancienneté $\rightarrow$ années
`amount_per_emp_year`	montant / (1 + années)
`age_group`	18–25, 26–35, 36–50, 50+
`has_checking_account`	détient un compte courant

+ regroupement des modalités rares (< 2 %)

Résultat (régression logistique, AUC CV)

sans FE	0,771 ± 0,053
avec FE	0,768 ± 0,044

À retenir

Pas de gain ici : les variables créées sont redondantes (la mensualité combine montant et durée). Une variable « intelligente » doit être **validée**, pas présumée utile.

Sélection de variables : pourquoi et comment ?

Pourquoi réduire ?

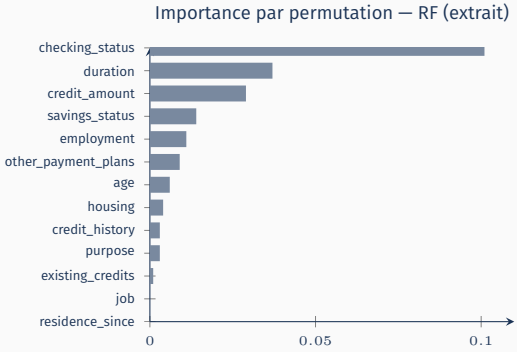
- modèle plus **simple à expliquer** et à documenter
- moins de **flux de données** à maintenir en production
- moins de risque d'**overfitting**
- entraînement plus rapide

Famille	Principe	Exemples
Filtre	score statistique variable par variable, sans modèle	corrélation, χ^2, information mutuelle, **IV**
Wrapper	on entraîne le modèle sur différents sous-ensembles	RFE, sélection forward / backward
Embarquée	la sélection a lieu pendant l'entraînement	pénalité L1 (Lasso), importance des arbres

🏛 Information Value (IV) — classique du scoring

Pouvoir discriminant d'une variable : $< 0{,}02$ inutile · $0{,}02$–$0{,}1$ faible · $0{,}1$–$0{,}3$ moyen · $> 0{,}3$ fort.

Sélection de variables dans le Lab 1

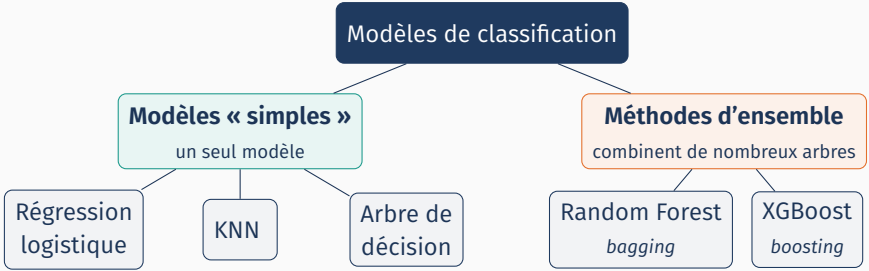


Méthode	Colonnes	AUC CV
Toutes les colonnes	68	0,768
RFECV (wrapper)	36	0,777
L1, $C = 0{,}1$ (embarquée)	22	0,760
L1, $C = 0{,}05$	10	0,745
SelectKBest $k = 15$ (filtre)	15	0,744

À retenir

Un **noyau commun** ressort : compte courant, durée, montant, épargne. On peut **diviser par 2 à 3** le nombre de colonnes pour une perte d'AUC faible.

Cinq modèles, deux grandes familles



Pour chaque modèle, retenir

l'intuition · les hyperparamètres clés · forces et limites · usage bancaire

À retenir

Pas de meilleur modèle universel (« *no free lunch* ») : on **compare** sur ses données.

Régression logistique (1/2) : l'intuition

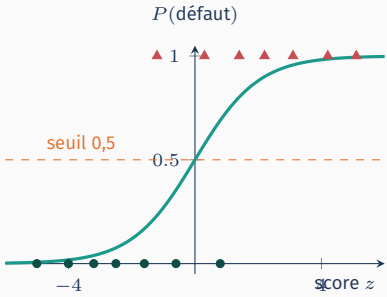
1. Un score linéaire : chaque variable pèse selon un coefficient

$$z = \beta_0 + \beta_1 \cdot \text{durée} + \beta_2 \cdot \text{montant} + \beta_3 \cdot \text{débiteur} + \dots$$

2. Transformé en probabilité par la fonction sigmoïde

$$P(\text{défaut}) = \frac{1}{1 + e^{-z}} \in [0, 1]$$

3. Apprentissage : on cherche les coefficients β qui rendent les défauts observés les plus probables (maximum de vraisemblance).



Régression logistique (2/2) : interprétation & usage

Odds ratio $= e^{\beta}$: facteur multiplicatif de la *cote* du défaut quand la variable augmente d'une unité

- $e^{\beta} > 1$: augmente le risque
- $e^{\beta} < 1$: diminue le risque

Régularisation (hyperparamètre C = inverse de la force)

- **L2** (Ridge) : rétrécit tous les coefficients $\rightarrow$ stabilité
- **L1** (Lasso) : met des coefficients à **zéro** $\rightarrow$ sélection

🏛 Base des **scorecards** : coefficients convertis en **points**.

Forces

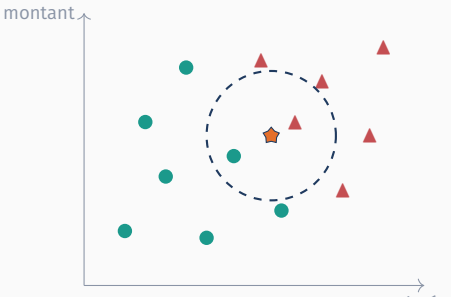
Très **explicable** · stable · rapide · probabilités bien calibrées · standard réglementaire

Limites

Relations **linéaires** uniquement (sauf feature engineering) · sensible aux variables corrélées · nécessite encodage et scaling

Lab 1 : $C = 0{,}03$, pénalité L2 — AUC test **0,802** — **modèle retenu**

K plus proches voisins (KNN)



$k = 5$: 3 défauts, 2 bons
$\Rightarrow$ P(défaut) = 60 %

Idée : « dis-moi qui sont tes voisins… »

1. calculer la **distance** entre le nouveau client et tous les clients connus
2. garder les k plus proches
3. prédire la classe **majoritaire** (ou la proportion)

Hyperparamètres : k, distance (euclidienne, Manhattan), pondération par la distance

Forces

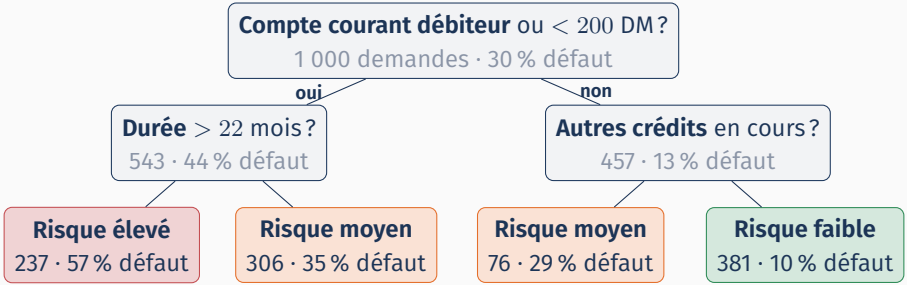
Intuitif · aucun entraînement · non linéaire

Limites

Lent en prédiction · scaling indispensable · souffre du grand nombre de variables

Usage bancaire : moteurs de recommandation (« clients similaires »), rarement pour l'octroi.

Arbre de décision (1/2) : des règles lisibles



Arbre construit à la main pour l'illustration, avec les taux de défaut réels du jeu German Credit (gauche = oui, droite = non).

À retenir

L'arbre découpe la population par une suite de questions oui / non. Chaque feuille donne une **probabilité** = taux de défaut des clients qui y tombent.

Arbre de décision (2/2) : comment choisir les questions ?

À chaque nœud, on choisit la variable et le seuil qui rendent les groupes les plus **purs**.

Impureté de Gini d'un groupe :

$$G = 1 - p_{\text{défaut}}^2 - p_{\text{bon}}^2$$

- 50 % / 50 % $\rightarrow G = 0{,}5$ (mélange maximal)
- 100 % / 0 % $\rightarrow G = 0$ (groupe pur)

Alternative : l'**entropie**.

Hyperparamètres : profondeur maximale, nb minimal de clients par feuille, critère

Forces

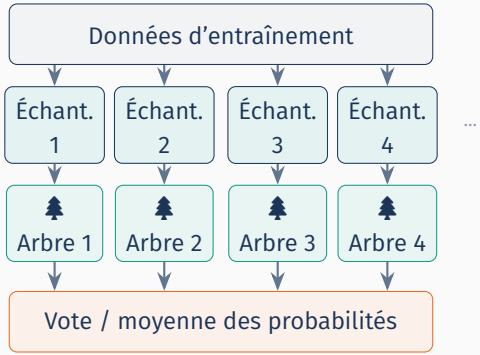
Très lisible · capte les non-linéarités et interactions · pas de scaling · gère les catégories

Limites

Overfitting s'il est trop profond · **instable** (petit changement de données = autre arbre) · performance moyenne

Lab 1 : sans limite, AUC train **1,00** vs validation **0,64** (profondeur 13) ; limité à 4 niveaux : 0,84 vs 0,67.

Random Forest : la sagesse des foules



1. **Bagging** : chaque arbre apprend sur un échantillon tiré au hasard (avec remise)
2. à chaque nœud, seul un **sous-ensemble aléatoire de variables** est considéré
3. les arbres **votent**

Des arbres différents font des erreurs différentes, qui **se compensent**.

Hyperparamètres : nb d'arbres, profondeur, `min_samples_leaf`, `max_features`

Forces

Robuste · peu de réglages · bonne performance · importance des variables

Limites

Moins explicable (« boîte grise ») · modèle volumineux

Gradient Boosting & XGBoost : apprendre de ses erreurs



Chaque nouvel arbre (petit) se concentre sur ce que les précédents **prédisent mal**.

Hyperparamètres clés

- `n_estimators` (nb d'arbres) et `learning_rate` (taille du pas) : à régler **ensemble**
- `max_depth` (2 à 6), `subsample`, `colsample_bytree`
- régularisation (`reg_lambda`), *early stopping*

Forces

Souvent **la meilleure performance** sur données tabulaires · gère manquants et non-linéarités

Limites

Nombreux hyperparamètres · risque d'overfitting · explicabilité via SHAP

🏛 Cas bancaire

Très utilisé en **fraude**, AML, churn ; de plus en plus en crédit avec SHAP. Variantes : LightGBM, CatBoost.

Comparatif des modèles

Modèle	Performance	Explicabilité	Scaling	Usage bancaire typique
Régression logistique	★★★	★★★★★	oui	scorecards d'octroi, PD réglementaire
KNN	★★	★★★	oui	recommandation, similarité
Arbre de décision	★★	★★★★★	non	règles métier, segmentation de risque
Random Forest	★★★★	★★	non	fraude, churn, benchmark
XGBoost	★★★★★	★★	non	fraude, AML, churn, octroi (+ SHAP)

À retenir

Stratégie fréquente en banque : un modèle complexe comme **challenger** (benchmark de performance) et un modèle explicable comme **champion** si l'écart est faible.

La matrice de confusion

		Prédit bon	Prédit défaut
Réel bon		**VN** bon payeur accepté	**FP** bon payeur refusé ✗
Réel défaut		**FN** défaut accepté ✗	**VP** défaut détecté

Classe positive = **défaut** (l'événement à détecter)

- **FN** : perte en capital (coût élevé)
- **FP** : manque à gagner, client mécontent

🏛 Cas bancaire

Fraude : FN = fraude non bloquée ; FP = carte d'un client honnête bloquée.
AML : FN = blanchiment non détecté ; FP = alerte à traiter par un analyste.

Accuracy, précision, rappel, F1

Accuracy $\dfrac{VP + VN}{\text{total}}$ % de bonnes réponses

Précision $\dfrac{VP}{VP + FP}$ fiabilité des alertes

Rappel $\dfrac{VP}{VP + FN}$ part des défauts détectés

F1 $2 \cdot \dfrac{\text{précision} \times \text{rappel}}{\text{précision} + \text{rappel}}$

Le paradoxe de l'accuracy

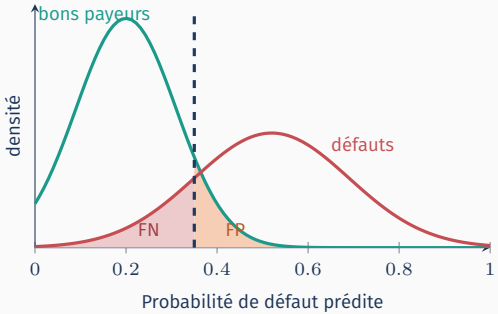
Un modèle qui **accepte tout le monde** :
accuracy = **70 %** (German Credit)
rappel = **0 %** : aucun défaut détecté !

En fraude (0,2 % de fraudes) : accuracy de 99,8 % pour un modèle inutile.

À retenir

Précision et rappel évoluent **en sens inverse** quand on déplace le seuil : arbitrage métier.

Le seuil de décision : un choix métier

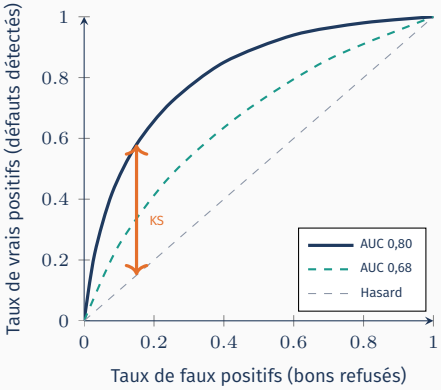


- Le modèle produit une **probabilité** ; la décision dépend du **seuil**
- Seuil **bas** : plus de défauts détectés, mais plus de bons clients refusés
- Seuil **haut** : plus d'acceptations, mais plus de pertes
- Le seuil 0,5 n'a **rien de naturel**

🏛 Cas bancaire

Politique d'octroi en 3 zones : **accepter** · **revoir** (étude manuelle) · **refuser**.

Courbe ROC, AUC, Gini et KS



- La courbe ROC montre tous les compromis possibles, **pour tous les seuils**
- **AUC** : probabilité qu'un défaut ait un score plus élevé qu'un bon payeur (0,5 = hasard, 1 = parfait)
- **Gini** $= 2 \times \text{AUC} - 1$: l'indicateur de discrimination de référence en **risque de crédit**
- **KS** : écart maximal entre les distributions de scores des bons et des mauvais

Lab 1 — régression logistique (test)

AUC **0,802** Gini **0,603** KS **0,524**

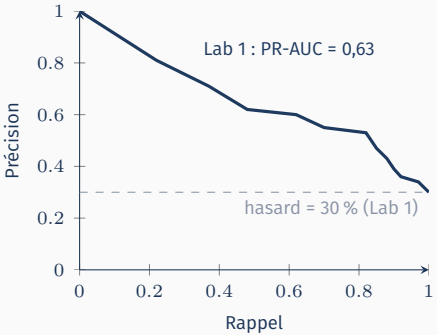
Ordre de grandeur courant pour un score d'octroi : Gini de 0,5 à 0,7.

Classes rares : la courbe précision-rappel

- Quand l'événement est **rare** (fraude : < 1 %), la ROC peut sembler excellente alors que les alertes sont majoritairement des fausses alertes
- La courbe **précision-rappel** et son aire (**PR-AUC**) sont plus parlantes
- Référence du hasard : la précision vaut le **taux d'événements**

🏛 Cas bancaire

Fraude carte : « sur 100 alertes, combien de vraies fraudes ? » (précision) et « quelle part des fraudes est bloquée ? » (rappel).



Choisir le seuil par le coût métier

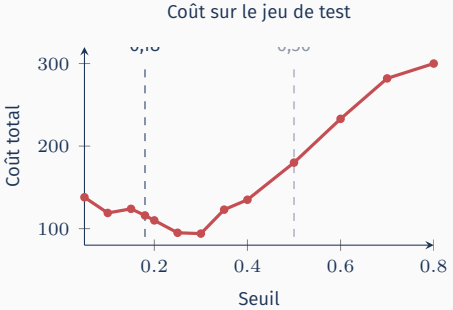
$$\text{Coût} = 5 \times FN + 1 \times FP$$

Seuil choisi sur les prédictions **hors-pli du train** (jamais sur le test !), puis appliqué au test.

Théorie : pour des probabilités bien calibrées,

$$\text{seuil optimal} = \frac{C_{FP}}{C_{FP} + C_{FN}} = \frac{1}{6} \approx 0{,}17$$

Lab 1 (test)	Seuil 0,50	Seuil 0,18
Défauts détectés	26 / 60	**54 / 60**
Bons refusés	10	86
Taux d'acceptation	82 %	**30 %**
Coût total	180	**116**



À retenir

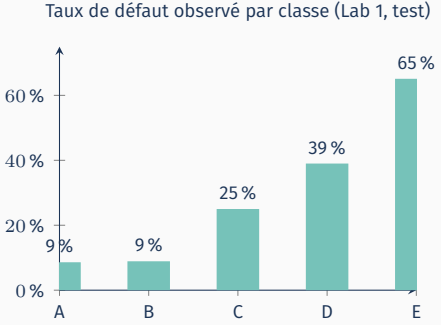
Le minimum **sur le test** (0,30) diffère du seuil choisi sur le train (0,18) : le choisir sur le test serait une **fuite**. Et l'acceptation s'effondre : en pratique, optimisation **sous contrainte commerciale** + revue manuelle.

De la probabilité à la grille de risque

- La PD est traduite en **classes de risque** (A à E), plus faciles à utiliser pour l'octroi, la tarification et le pilotage
- Bornes fixées sur le train (quintiles de PD), appliquées au test
- Une bonne grille est **monotone** : le taux de défaut croît de A à E
- Avec 40 dossiers par classe, de petites inversions relèvent du bruit

Cas bancaire

Base de la tarification au risque et du calcul de la **perte attendue** : $EL = PD \times LGD \times EAD$.



Et pour la régression ?

MAE	$\frac{1}{n}\sum \lvert y_i - \hat{y}_i \rvert$	erreur moyenne
RMSE	$\sqrt{\frac{1}{n}\sum (y_i - \hat{y}_i)^2}$	pénalise les grosses erreurs
MAPE	$\frac{1}{n}\sum \left\lvert \frac{y_i - \hat{y}_i}{y_i} \right\rvert$	erreur en %
R^2	$1 - \frac{\sum (y_i - \hat{y}_i)^2}{\sum (y_i - \bar{y})^2}$	variance expliquée

Cas bancaire

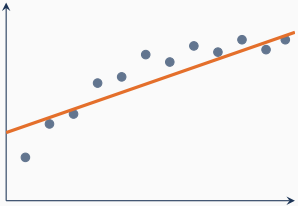
- **LGD** (perte en cas de défaut) : MAE, RMSE
- **Prévision de flux** de trésorerie : MAPE
- **Valorisation** d'un bien immobilier : MAPE, R^2

À retenir

Mêmes principes qu'en classification : pipeline, validation croisée, overfitting, tuning. Seules les **métriques** changent.

Underfitting, bon compromis, overfitting

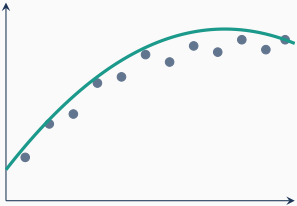
Underfitting



Modèle **trop simple** : il rate la tendance.

Erreur élevée en train ET en test.

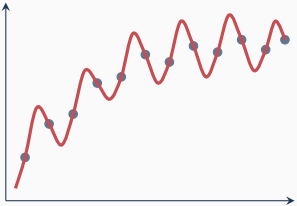
Bon compromis



Il capte la tendance, pas le bruit.

Bonne généralisation.

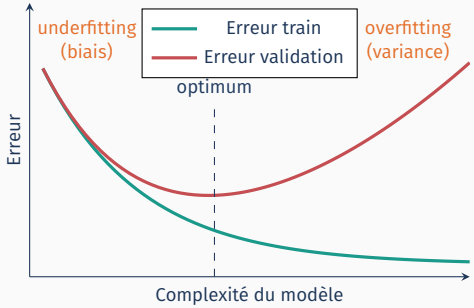
Overfitting



Il apprend **le bruit par cœur**.

Parfait en train, mauvais en test.

Le compromis biais-variance

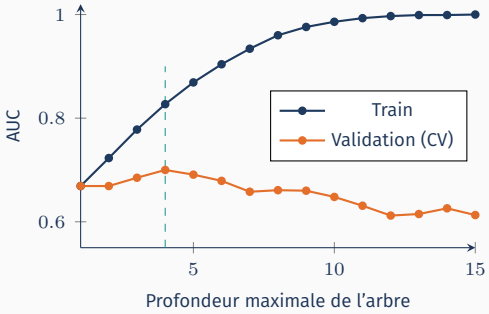


- **Biais** : erreur due à des hypothèses trop simples
- **Variance** : sensibilité aux particularités de l'échantillon d'entraînement
- Augmenter la complexité réduit le biais mais augmente la variance
- On cherche le **minimum de l'erreur de validation**

À retenir

L'écart entre score train et score validation est le **signal d'alerte** de l'overfitting.

Diagnostiquer : courbes de validation (Lab 1)



- L'AUC **train** monte jusqu'à 1 : l'arbre apprend chaque client par cœur
- L'AUC **validation** plafonne vers 4 niveaux, puis **baisse**
- Même diagnostic pour le KNN ($k = 1$: overfitting extrême) et la régression logistique (C trop grand)

Courbe d'apprentissage

Score en fonction de la **taille du train**. Si la validation progresse encore : plus de données aideraient. Si elle plafonne : changer de modèle ou de variables.

Les remèdes

Underfitting (trop simple)	**Overfitting** (trop complexe)
Ajouter des variables pertinentes (feature engineering)	Plus de données d'entraînement
Modèle plus flexible (arbres $\rightarrow$ ensembles)	**Régularisation** (L1 / L2, C plus petit)
Moins de régularisation	Limiter la complexité : profondeur, `min_samples_leaf`, nb de voisins k plus grand
Entraîner plus longtemps (boosting)	*Early stopping*, *learning rate* plus faible
	Sélection de variables, méthodes d'ensemble (bagging)

À retenir

Toujours diagnostiquer avec un jeu de **validation** (ou la validation croisée), jamais avec le jeu de test.

Paramètres vs hyperparamètres

Paramètres

appris par l'algorithme pendant l'entraînement

coefficients β, seuils des arbres

Hyperparamètres

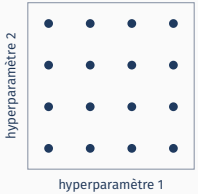
choisis avant l'entraînement ; ils contrôlent la complexité

profondeur, nb d'arbres, C, k

Modèle	Hyperparamètres clés
Régression logistique	`C`, pénalité L1 / L2, `class_weight`
KNN	`n_neighbors`, distance, pondération
Arbre de décision	`max_depth`, `min_samples_leaf`, critère
Random Forest	`n_estimators`, `max_depth`, `max_features`, `min_samples_leaf`
XGBoost	`learning_rate`, `n_estimators`, `max_depth`, `subsample`, `reg_lambda`

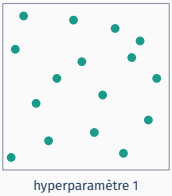
Stratégies de recherche

Grid search



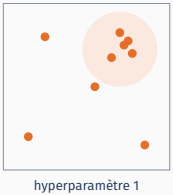
Toutes les combinaisons d'une grille. Exhaustif mais coûteux.

Random search



N combinaisons tirées au hasard. Efficace sur de grands espaces.

Bayésienne (Optuna)



Explore les zones prometteuses d'après les essais précédents.

À retenir

Chaque combinaison est évaluée en **validation croisée** sur le train, avec la **métrique métier** choisie (Lab 1 : AUC).

Résultats du tuning — Lab 1

	AUC CV		AUC
Modèle	**défaut**	**optimisé**	**test**
Random Forest	0,794	**0,799**	0,812
XGBoost	0,778	0,793	0,809
Régression logistique	0,768	0,779	0,802
KNN	0,737	0,778	0,803
Arbre de décision	0,617	0,734	0,728

- Le tuning aide surtout les modèles **sensibles à la complexité** (arbre : +0,12 ; KNN : +0,04)
- Une fois réglés, les meilleurs modèles sont **très proches**
- Écart RF / régression logistique : 0,020, **inférieur** à l'écart-type de la CV (0,035)

À retenir

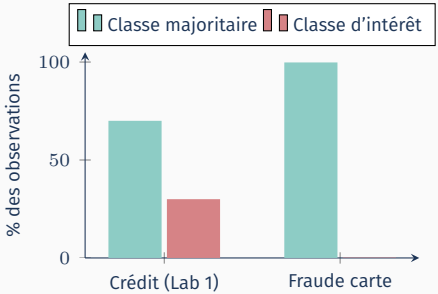
⇒ On retient la **régression logistique** : performance équivalente, explicabilité et stabilité supérieures.

Classes déséquilibrées

Défaut (≈ 2–30 %), fraude (< 1 %), blanchiment (très rare) : la classe intéressante est **minoritaire**.

Leviers

- **Pondérer** les classes : `class_weight="balanced"`, `scale_pos_weight`
- **Rééchantillonner** : sous-échantillonner la majorité, sur-échantillonner la minorité (SMOTE) — **dans le pipeline**, sur le train seulement
- **Ajuster le seuil** de décision
- Choisir les bonnes **métriques** : AUC, PR-AUC, rappel, coût



Attention

Pondération et rééchantillonnage **déforment les probabilités** : recalibrer si la PD est utilisée telle quelle (tarification, provisions).

Expliquer un modèle : global et local

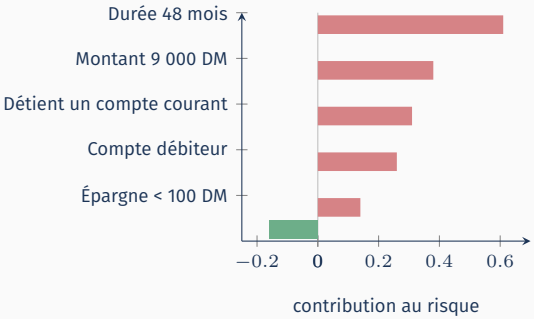
Explicabilité globale — le modèle

- **Coefficients / odds ratios** (régression logistique)
- **Importance par permutation** : chute de performance quand on mélange une variable — fonctionne pour tout modèle
- **Dépendance partielle** : effet moyen d'une variable sur la prédiction

Explicabilité locale — une décision

- **SHAP** : contribution de chaque variable à **cette** prédiction, par rapport à la moyenne
- motifs de refus communiqués au client

Exemple d'explication locale (application Lab 1)



PD = 60 % · classe E · **Refuser**

Équité : mesurer avant de déployer

- Variables **sensibles** : sexe, nationalité, âge, origine…
- Les retirer (« *fairness through unawareness* ») est **nécessaire mais pas suffisant** : d'autres variables servent de **proxy**
- Il faut **mesurer** : taux de refus, PD moyenne, performance par groupe
- et **documenter** les écarts et leur justification

Lab 1

La régression logistique utilisait `personal_status` (sexe) : « homme célibataire » apparaissait comme protecteur.

Lab 1, test	Défaut	Refus	AUC
Femmes (61)	32,8 %	75 %	0,83
Hommes (139)	28,8 %	68 %	0,79

Variables sensibles	AUC CV
incluses	0,779
retirées	0,772

À retenir

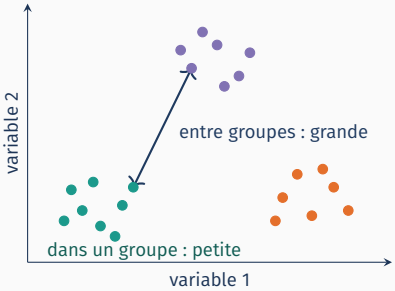
Retirer les variables sensibles ne coûte presque rien ici ; écarts à suivre sur de plus grands volumes.

Récapitulatif : la check-list du modèle supervisé

- ☑ Cible et coût des erreurs définis avec le métier
- ☑ Données disponibles au moment de la décision
- ☑ EDA confrontée à l'expertise métier
- ☑ Toutes les transformations dans un **pipeline**
- ☑ Feature engineering **validé** en CV
- ☑ Plusieurs modèles comparés en **validation croisée**
- ☑ Overfitting diagnostiqué (train vs validation)
- ☑ Hyperparamètres réglés en CV, test utilisé **une fois**
- ☑ Métriques adaptées : AUC / Gini / KS, rappel, coût
- ☑ Seuil choisi avec le métier, hors jeu de test
- ☑ Explicabilité et équité mesurées et documentées
- ☑ Plan de monitoring (PSI) et de réentraînement

Apprentissage non supervisé : clustering

Le clustering : regrouper sans étiquette



Objectif : des groupes **homogènes** (clients proches) et **distincts** (groupes éloignés).

Pas de cible : l'algorithme propose, le **métier dispose**.

Un bon segment est

interprétable · actionnable · de taille suffisante · stable dans le temps

🏛 Cas bancaire

Segmentation marketing ou de risque, détection d'atypiques (fraude, AML).

Tout repose sur la notion de distance

Euclidienne	$\sqrt{\sum_j (x_j - y_j)^2}$	numériques
Manhattan	$\sum_j \lvert x_j - y_j \rvert$	numériques
Hamming	nb de modalités différentes	catégorielles
Gower	moyenne de distances normalisées	**mixtes**

Gower, variable par variable

numérique : $\frac{|x_j - y_j|}{\text{étendue}_j}$ catégorielle : 0 si identique, 1 sinon

Exemple : deux clients

	Client A	Client B	écart
Âge	30	45	15 ans
Solde	500 €	2 500 €	2 000 €
Métier	cadre	cadre	0
Crédit immo	oui	non	1

Sans mise à l'échelle…

…le solde (2 000) écrase complètement l'âge (15) dans la distance euclidienne. **Standardiser est indispensable.**

Préparer les données pour le clustering (Lab 2)

Bank Marketing (UCI) : 45 211 clients

- numériques : `age`, `balance` (solde moyen)
- catégorielles : métier, statut marital, études, incident de crédit, crédit immobilier, crédit personnel
- la souscription au dépôt à terme (`y`) est **mise de côté** : elle servira à évaluer les segments

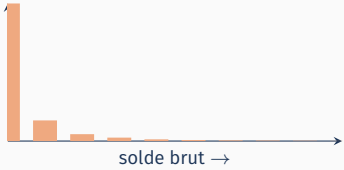
Étapes

- 12 métiers $\rightarrow$ 5 groupes socio-professionnels
- solde : transformation **Yeo-Johnson** (accepte les négatifs) puis standardisation
- échantillons pour les calculs en $O(n^2)$

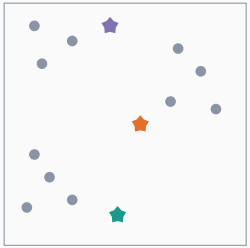
Le solde, une variable très asymétrique

8,3 % de soldes négatifs · max 102 127 € asymétrie (skewness) : **8,4** $\rightarrow$ **1,1** après Yeo-Johnson

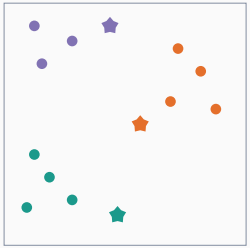
quelques clients très aisés « tirent » la distribution (schéma)



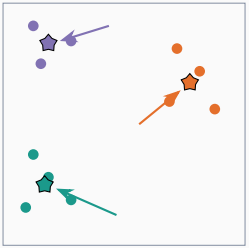
K-Means : l'algorithme pas à pas



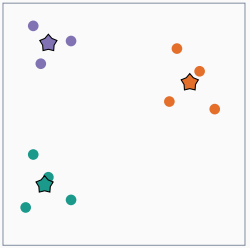
1. Placer k centres au hasard



2. Affecter chaque client au centre le plus proche



3. Recalculer les centres (moyenne du groupe)

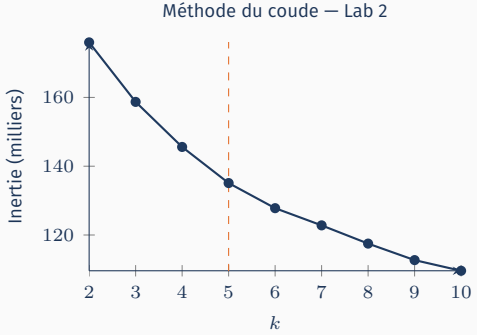


4. Répéter 2–3 jusqu'à stabilité

K-Means minimise l'**inertie** : somme des distances au carré entre chaque client et le centre de son groupe.

K-Means++ place des centres initiaux bien répartis ; on relance plusieurs fois (`n_init`) et on garde la meilleure solution.

Choisir le nombre de groupes k



k	Silhouette ↑	Davies-Bouldin ↓	Calinski-H. ↑
2	0,165	2,09	9 224
3	**0,169**	1,82	7 581
4	0,157	1,81	6 863
5	0,150	1,91	6 427
6	0,143	1,91	5 949
8	0,150	1,71	5 184

À retenir

Pas de coude net, silhouette faible : normal sur des données clients (continuum). Les métriques **guident**, le métier **tranche** : assez de segments pour différencier les offres, pas trop pour rester actionnable $\Rightarrow k = 5$.

Les métriques internes du clustering

Silhouette d'un client i

$$s(i) = \frac{b(i) - a(i)}{\max(a(i), b(i))} \in [-1, 1]$$

- $a(i)$: distance moyenne aux clients de **son** groupe
- $b(i)$: distance moyenne au groupe **voisin le plus proche**
- proche de 1 : bien classé · 0 : à la frontière · < 0 : mal classé



$a(i)$: petit $b(i)$: grand

Davies-Bouldin	ressemblance entre groupes voisins (↓)
Calinski-Harabasz	variance inter / intra (↑)
Stabilité (ARI)	mêmes groupes sur d'autres échantillons ?

Forces et limites de K-Means

Forces

Simple · rapide · passe à l'échelle (45 000 clients en quelques secondes) · affecte facilement les **nouveaux clients**

Limites

- choisir k à l'avance
- groupes supposés « ronds » et de tailles comparables
- sensible aux valeurs extrêmes et à l'échelle
- conçu pour des variables **numériques** : la « moyenne » d'une variable catégorielle a peu de sens

Constat du Lab 2 (one-hot + K-Means)

Les groupes se découpent selon des variables **binaires** très présentes :

- un segment à **100 %** de crédits immobiliers
- un segment à **0 %**
- un segment à **97 %** de diplômés du supérieur

Les colonnes 0/1 « tirent » les centres.

À retenir

Pour des données **mixtes** (numériques + catégorielles) : **K-Prototypes**.

K-Prototypes : le clustering des données mixtes

Combinaison de **K-Means** (numériques : moyenne, distance euclidienne) et de **K-Modes** (catégorielles : mode, distance de Hamming) :

$$d(\text{client}, \text{prototype}) = \underbrace{\sum_{\text{num}} (x_j - \mu_j)^2}_{\text{partie numérique}} + \gamma \times \underbrace{\sum_{\text{cat}} \mathbb{1}[x_j \neq \text{mode}_j]}_{\text{nb de modalités différentes}}$$

- γ règle le **poids des catégorielles**
- par défaut : $\gamma = 0{,}5\times$ écart-type moyen des numériques
  (Lab 2 : $\gamma = 0{,}48$)
- prototypes **directement lisibles**

Huang (1998). Implémentation : package Python `kmodes`.

Prototypes du Lab 2 (âge et solde re-transformés)

Seg.	Âge	Solde	Groupe pro.	Statut	Immo
0	53	859 €	Employés	marié	non
1	42	8 484 €	Cadres	marié	non
2	29	593 €	Employés	célibataire	oui
3	38	619 €	Employés	marié	oui
4	40	−661 €	Employés	marié	oui

L'effet de γ : un hyperparamètre métier

γ	Tailles (éch. de 10 000)	Écart d'âge	Écart % immo
0,1	3 400 · 2 850 · 2 780 · 692 · 278	8,9	0,15
0,5	2 890 · 2 363 · 2 108 · 2 014 · 625	9,8	0,21
1,5	3 625 · 1 993 · 1 944 · 1 838 · 600	9,0	0,23
3,0	3 424 · 2 486 · 1 794 · 1 424 · 872	8,2	**0,35**

Écarts = écart-type, entre segments, de l'âge moyen et de la part de crédit immobilier.

- γ **faible** : segmentation pilotée par l'âge et le solde
- γ **élevé** : les catégorielles dominent (écart de crédit immo × 2)

À retenir

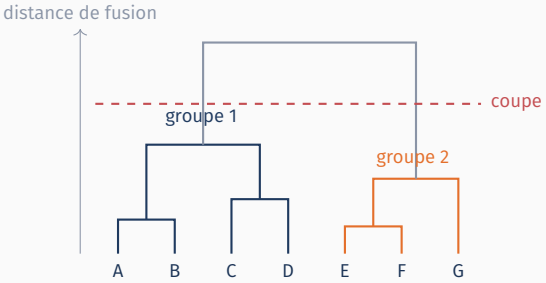
Choisir γ, c'est choisir **ce qui doit différencier** les segments : une décision métier.

Classification ascendante hiérarchique (CAH)

1. au départ, chaque client est un groupe
2. on **fusionne** les deux groupes les plus proches
3. on répète jusqu'à n'avoir plus qu'un groupe

Le résultat est un **dendrogramme** que l'on « coupe » à la hauteur voulue : **pas besoin de fixer** k à l'avance.

La hauteur d'une fusion mesure à quel point les groupes fusionnés sont **différents** : on coupe là où les branches sont longues.



Le critère d'agrégation change tout

Linkage	Distance entre deux groupes
Ward	hausse d'inertie si on fusionne → groupes compacts, équilibrés (euclidien)
Complet	distance **maximale** entre membres → compacts
Moyen	distance **moyenne** → intermédiaire
Simple	distance **minimale** → effet de chaîne

Lab 2 — 3 000 clients, 5 groupes

Ward (one-hot)	1 235 · 633 · 546 · 335 · 251
Gower + complet	1 365 · 1 164 · 338 · 94 · 39
Gower + moyen	**1 922** · 943 · 106 · 20 · 9

À retenir

Le linkage moyen isole des **micro-groupes atypiques** : inutile pour le marketing, utile pour **détecter des profils atypiques**.

La CAH coûte $O(n^2)$ en mémoire : idéale pour **explorer** un échantillon, pas pour segmenter 45 000 clients ni affecter de nouveaux clients.

Comparer les approches (Lab 2)

Méthode	Silhouette (Gower)	Plus petit segment	Plus grand segment
K-Means (one-hot)	0,136	138	817
K-Prototypes	0,014	66	1 109
CAH Ward (one-hot)	0,119	251	1 235
CAH Gower complet	**0,203**	39	1 365
CAH Gower moyen	0,164	9	1 922

Évaluation sur le même échantillon de 3 000 clients.

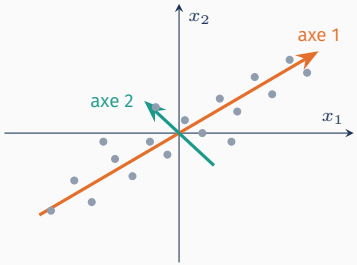
Stabilité de K-Prototypes (ARI sur d'autres échantillons) : **0,47 à 0,59**.

- Les méthodes ne trouvent **pas les mêmes** segments
- Les métriques dépendent de la distance utilisée : K-Prototypes n'optimise pas Gower
- La meilleure silhouette va de pair avec un segment de 39 clients

À retenir

On retient **K-Prototypes** pour des raisons **métier** : données mixtes, prototypes lisibles, tailles exploitables, affectation des nouveaux clients.

Réduction de dimension : l'ACP pour visualiser



- L'**ACP** (analyse en composantes principales) cherche de nouveaux axes, combinaisons des variables, qui conservent le **maximum de variance**
- On projette les clients sur les 2 premiers axes pour **visualiser** les segments
- Autres usages : compression, débruitage, décorrélation avant un modèle

Lab 2

20 colonnes $\rightarrow$ 2 axes conservant **25 % + 21 %** de la variance : la projection 2D est une vue **partielle** des segments.

Méthodes non linéaires : t-SNE, UMAP (visualisation).

Profiler les segments : la vraie valeur ajoutée

1. Décrire chaque segment par des **indicateurs métier** : taille, âge, solde médian, % crédit immobilier, % découverts…
2. Comparer chaque segment à la **moyenne du portefeuille**
3. Mesurer l'**appétence** avec une variable non utilisée pour segmenter : **lift** = taux du segment / taux moyen
4. **Nommer** les segments (personas) et définir les **actions**

Les numéros de segments sont arbitraires

Ils changent à chaque entraînement. Dans le Lab 2, les noms sont attribués par des **règles métier** sur les profils :

1. solde médian maximal → Patrimoniaux
2. part de découverts maximale → Clients fragiles
3. âge moyen maximal → Seniors
4. âge moyen minimal → Jeunes actifs
5. restant → Familles propriétaires

Les cinq personas du Lab 2

Persona	Clients	Âge	Solde méd.	Immo	Perso	Découvert	Souscription (lift)
Familles propriétaires	37 %	39	383 €	70 %	16 %	7 %	9,0 % (0,77)
Seniors	28 %	54	518 €	35 %	16 %	5 %	12,8 % (1,09)
Jeunes actifs	26 %	29	355 €	59 %	16 %	8 %	14,1 % (1,21)
Patrimoniaux	7 %	44	7 296 €	44 %	7 %	0 %	**15,2 % (1,30)**
Clients fragiles	3 %	41	−537 €	81 %	36 %	100 %	5,6 % (0,48)

Cibler

Campagnes d'épargne vers les segments au **lift > 1** : autant de souscriptions avec moins d'appels.

Accompagner

Clients fragiles : accompagnement budgétaire, regroupement de crédits, **suivi du risque**.

Adapter l'offre

Familles : protection (prévoyance, assurances) ; Patrimoniaux : gestion de patrimoine.

Industrialiser une segmentation

- **Affectation** des nouveaux clients avec le modèle figé (`predict`) : quotidienne ou mensuelle
- **Suivi** de la répartition entre segments et des distributions des variables (**PSI**)
- **Re-segmentation** périodique (ex. annuelle) ou déclenchée par une dérive durable
- Les segments doivent rester **stables** : les équipes métier construisent leurs offres dessus

Lab 2 — lot simulé après une campagne digitale

75 % de clients de moins de 35 ans (contre 33 % en référence) :
PSI de la répartition des segments = **0,46** 🔴

→ ce n'est pas forcément un problème du **modèle** mais une **information métier** : le mix client change.

À retenir

Dans l'application du Lab 2, la page « Qualité & dérive » permet de simuler ces changements.

Labs, applications & conclusion

Lab 1 — Credit scoring (classification)

Objectif : prédire la probabilité de défaut d'un demandeur de crédit et proposer une décision.

Données : German Credit (UCI) — 1 000 demandes, 20 variables

Notebook `lab1_credit_scoring.ipynb`

1. Cadrage métier & matrice de coût
2. EDA
3. Preprocessing, pipeline, démonstration du leakage
4. Feature engineering
5. Baseline des 5 modèles en validation croisée
6. Sélection de variables (filtre, RFE, L1, permutation)
7. Overfitting : courbes de validation et d'apprentissage
8. Fine-tuning (Grid / Random search)
9. Évaluation : ROC, KS, seuil par le coût, grille de risque
10. Équité & explicabilité
11. Production : export, API, monitoring PSI

Résultats clés

- Modèle retenu : **régression logistique** régularisée ($C = 0{,}03$)
- AUC test **0,802** · Gini **0,603** · KS **0,524**
- Random Forest : AUC 0,812, écart non significatif
- Seuil optimal (coût 5 :1) : **0,18** — coût 180 $\rightarrow$ 116
- Grille A $\rightarrow$ E : défaut de 9 % à 65 %

À retenir

Messages : pipeline contre la fuite de données, AUC $\neq$ décision, seuil = choix métier, explicabilité.

Lab 2 — Segmentation client (clustering)

Objectif : construire des segments clients homogènes et actionnables.

Données : Bank Marketing (UCI) — 45 211 clients, profil socio-démographique et bancaire

Notebook `lab2_segmentation_clients.ipynb`

1. Cadrage métier
2. EDA (asymétries, V de Cramér)
3. Preprocessing (Yeo-Johnson, encodage, échantillonnage)
4. Feature engineering (groupes socio-pro., nb de crédits)
5. K-Means : coude, silhouette, Davies-Bouldin, ACP
6. K-Prototypes : coût, effet de γ
7. CAH : Ward, distance de Gower, dendrogrammes
8. Comparaison & stabilité (ARI)
9. Profilage & personas
10. Production : affectation, suivi de la dérive

Résultats clés

- Modèle retenu : **K-Prototypes**, $k = 5$, $\gamma = 0{,}48$
- 5 personas : Familles propriétaires (37 %), Seniors (28 %), Jeunes actifs (26 %), Patrimoniaux (7 %), Clients fragiles (3 %)
- Appétence épargne : lift de **0,48** à **1,30**

À retenir

Messages : standardiser, choisir l'algorithme selon le type de données, les métriques guident mais le métier tranche.

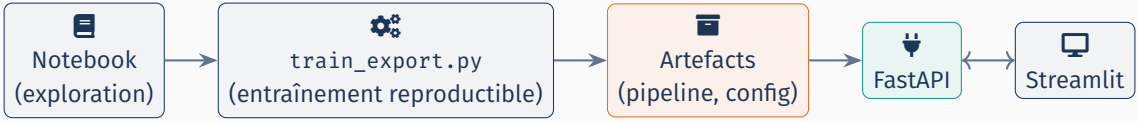
Les applications : du notebook au service

Credit scoring — API :8000 · UI :8501

- `POST /predict` : PD, score, classe A–E, décision *Accepter* / *Revoir* / *Refuser*, facteurs explicatifs
- `POST /predict-batch`, `GET /model-info`
- Pages : scoring d'une demande (jauge, facteurs, *what-if*) · scoring par lot · **dashboard KPIs** (acceptation, défaut, perte attendue) · **performance & monitoring** (ROC, coût/seuil, PSI)

Segmentation — API :8001 · UI :8502

- `POST /assign` : segment, persona, offres, proximité aux segments
- `POST /assign-batch`, `GET /segments`
- Pages : affecter un client (carte persona) · affectation par lot · **dashboard segments** (KPIs, treemap, profils, appétence, ACP) · **qualité & dérive** (PSI)



Mise en route

Structure du dépôt

```
Formation-ML-Banking/
  slides/      deck LaTeX Beamer
  labs/
    data/download_data.py
    lab1_credit_scoring.ipynb
    lab2_segmentation_clients.ipynb
  apps/
    credit_scoring/{backend,frontend}
    segmentation/{backend,frontend}
```

Installation & lancement

```
pip install -r labs/requirements.txt
pip install -r apps/requirements.txt
python labs/data/download_data.py

# Application credit scoring
cd apps/credit_scoring/backend
python train_export.py
uvicorn app:app --port 8000
cd ../frontend
streamlit run app.py --server.port 8501

# Segmentation : idem, ports 8001 / 8502
```

Ce qu'il faut retenir

1. **Le cadrage métier** conditionne tout : cible, coût des erreurs, contraintes
2. **Les données** font la performance : qualité, disponibilité au moment de la décision, feature engineering
3. **Le pipeline** protège de la fuite de données et devient l'objet déployé
4. **Comparer** plusieurs modèles en validation croisée ; le plus complexe n'est pas toujours le meilleur
5. **Les bonnes métriques** : AUC / Gini, rappel, coût — et un seuil choisi avec le métier
6. **L'overfitting** se diagnostique (train vs validation) et se corrige
7. **En non supervisé**, les métriques guident, l'utilité métier tranche
8. **Un modèle en production** se surveille (PSI), se réentraîne et se gouverne

À retenir

En banque, un bon modèle est **performant**, **explicable**, **équitable**, **documenté** et **surveillé**.

Pour aller plus loin

Ouvrages

- G. James et al., *An Introduction to Statistical Learning* (accès libre)
- A. Géron, *Hands-On Machine Learning with Scikit-Learn, Keras & TensorFlow*
- C. Molnar, *Interpretable Machine Learning* (accès libre)
- N. Siddiqi, *Credit Risk Scorecards*

Documentation & réglementation

- scikit-learn : guide utilisateur (pipelines, sélection de modèles)
- XGBoost, `kmodes`, SHAP, MLflow, FastAPI
- Règlement européen sur l'IA (AI Act)
- Orientations ABE sur l'octroi et le suivi des prêts
- FINMA, communication sur la surveillance 08/2024

Merci !

Questions & échanges

Place aux labs 